

PROJECT REPORT:

LIBRARY MANAGEMENT SYSTEM

- ❖ **Subject:** Software Engineering
- ❖ **Degree:** Bachelor of Cyber Security

➤ **TABLE OF CONTENTS**

1. Introduction
2. Objectives
3. Scope
4. Functional Requirements
5. Non-Functional Requirements
6. Use Case Diagram
7. Activity Diagram
8. Sequence Diagram
9. System Work Flow Diagram
10. Entity Relationship Diagram (ERD)
11. Validation and Verification
12. Software Implementation (Source Code)

1. Introduction

The Library Management System is a software application designed to help librarians manage the day-to-day operations of a library efficiently. In traditional libraries, record-keeping is often done manually using large physical registers. This manual process is slow, prone to human error, and makes searching for specific book records difficult.

This project introduces a computerized solution to store book details, manage student records, and track borrowed books. By digitizing these records, the system ensures that data is retrievable instantly, making the library management process faster and more reliable.

2. Objectives

The primary goals of this project are:

- To replace manual paper records with a secure digital database.
- To significantly reduce the time required to search for a book.
- To accurately track borrowed books, issued dates, and return deadlines.
- To provide a user-friendly interface for the librarian to add, update, or remove books.
- To minimize data loss and redundancy in library records.

3. Scope

The scope of this project defines the boundaries of the system:

- **Librarian (Admin):** Has full control over the system. The librarian can add new books to the inventory, register students, issue books, and accept returns.
- **Student (User):** The system allows searching for books to check availability (in future versions, students may have their own login).
- **Boundaries:** This version focuses on the backend database and the librarian's administrative interface. Online e-book reading and fine payment integration are outside the scope of this current version but may be added in future updates.

4. Functional Requirements

These are the specific behaviors and functions the software must perform:

1. **Login System:** The system must validate the librarian's username and password before granting access.
2. **Add Book:** The system must allow the entry of Book ID, Title, Author, and Quantity.
3. **Search Book:** The system must allow users to search for a book by its Title or ID.
4. **Issue Book:** The system must link a specific book to a student ID and update the status to "Issued."
5. **Return Book:** The system must accept returns and immediately update the inventory count.
6. **Display Inventory:** The system must generate a list of all books currently held in the library.

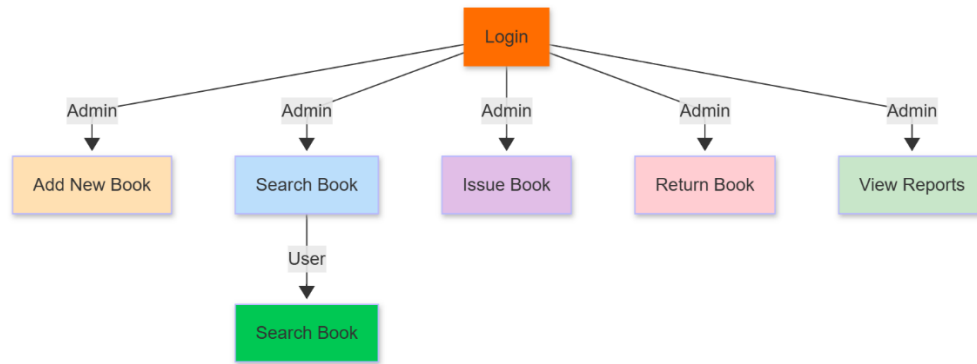
5. Non-Functional Requirements

These describe the quality and performance attributes of the system:

1. **Usability:** The interface is menu-driven and simple, requiring no advanced technical skills to operate.
2. **Performance:** Search queries must return results in less than 2 seconds.
3. **Reliability:** The system must ensure data persistence so that records are not lost after the program is closed.
4. **Security:** Critical functions (like deleting books) are restricted to authorized personnel only.

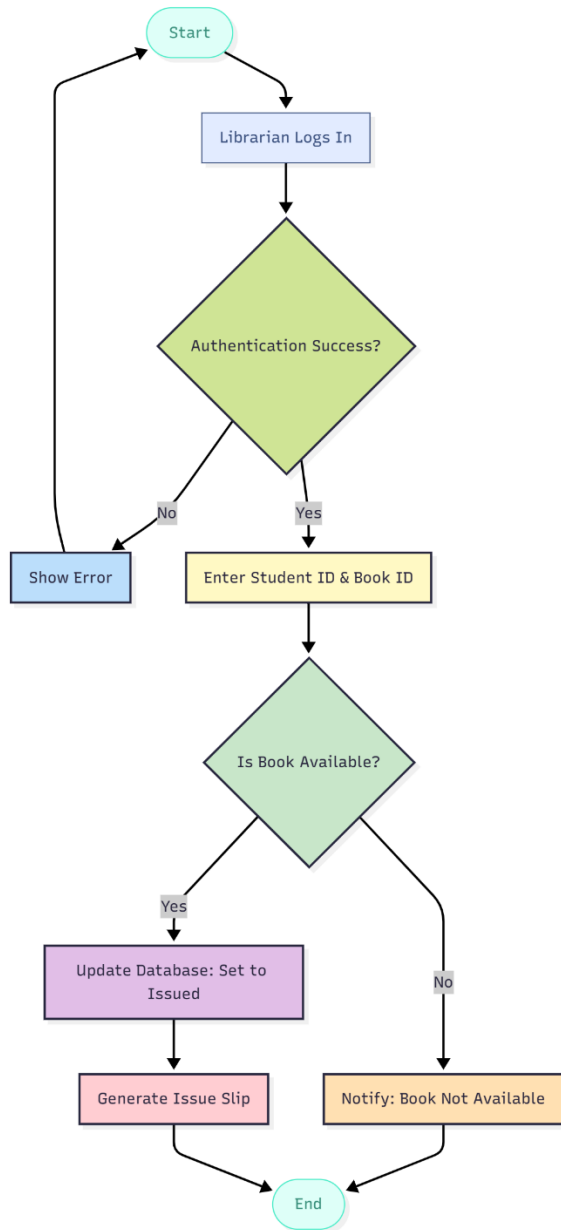
6. Use Case Diagram

- This diagram illustrates the interactions between the Librarian, the Student, and the System functions.



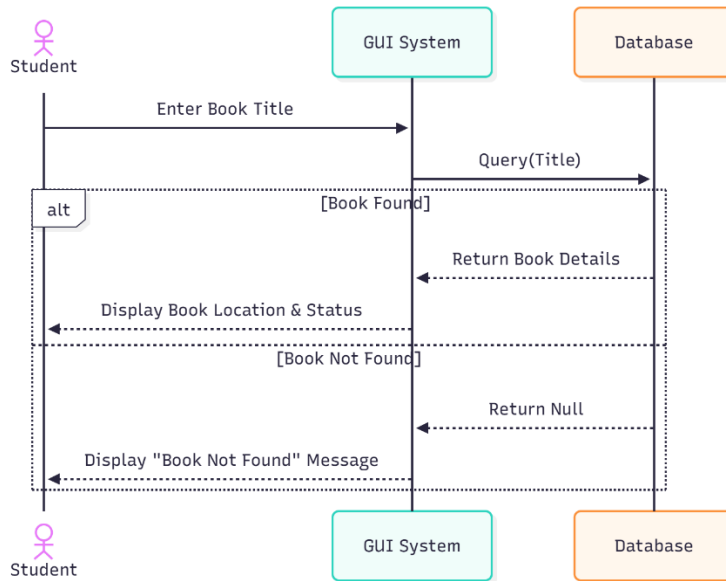
7. Activity Diagram

- This diagram represents the step-by-step workflow for the "Issue Book" process.



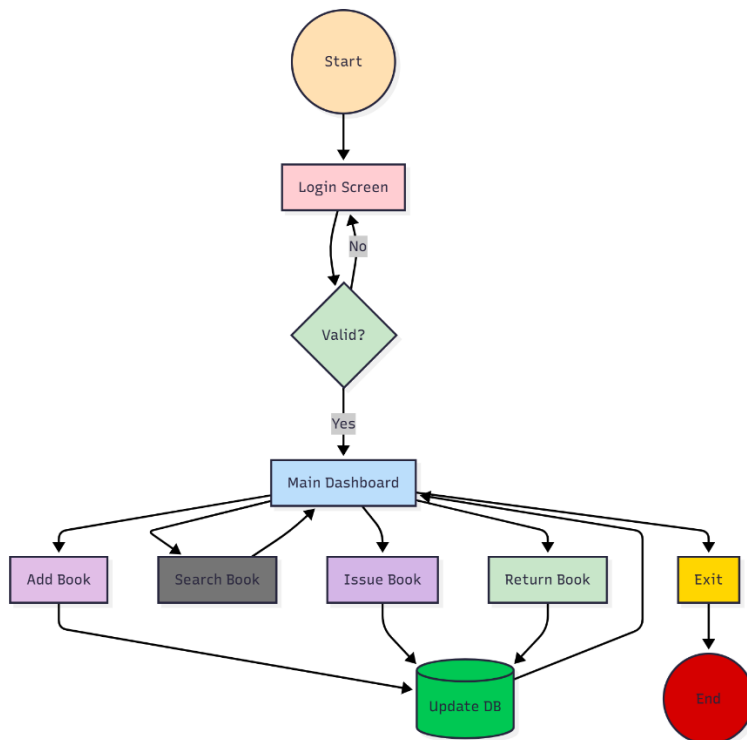
8. Sequence Diagram

- This diagram shows the timeline of messages exchanged between the User, the Interface, and the Database during a Book Search.



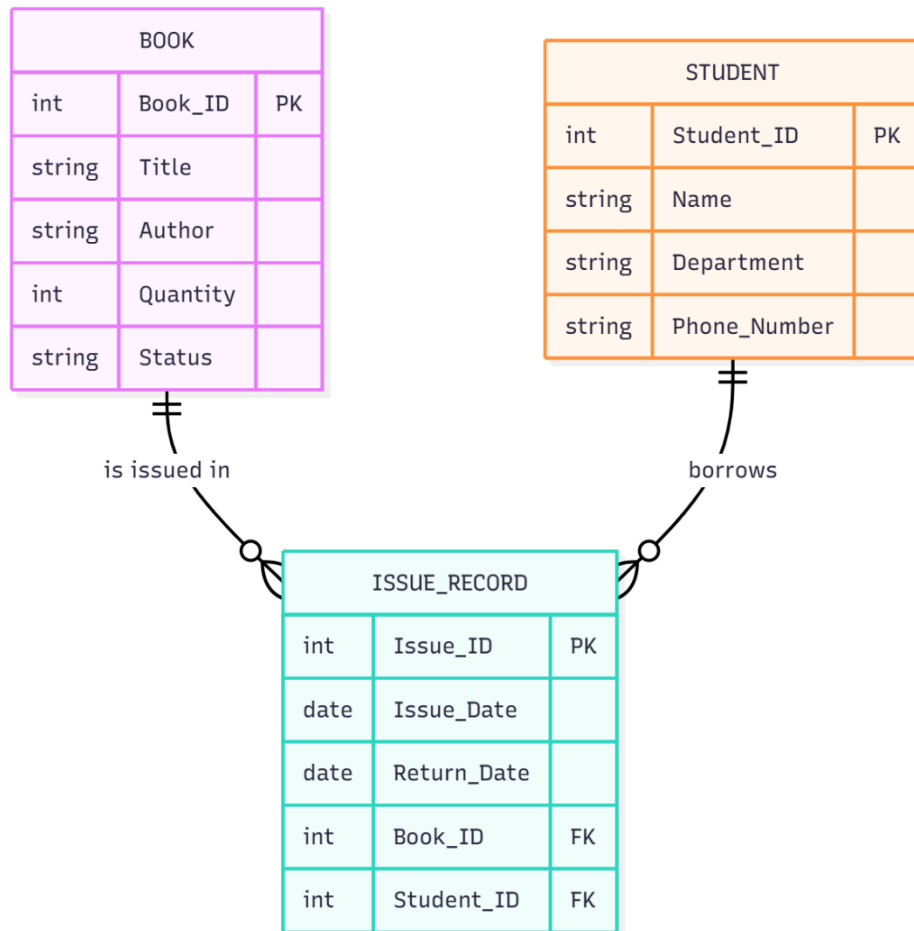
9. System Work Flow Diagram

- This diagram shows the overall navigation structure of the application from Login to Exit.



10. Entity Relationship Diagram (ERD)

- This diagram visualizes the database structure, showing the relationship between Books, Students, and Issue Records.



11. Validation and Verification

✓ **Verification**

- Are we building the product right?
- During the development phase, Unit Testing was performed. Each function (such as addBook and searchBook) was tested individually to ensure it stored data correctly in the variables.

✓ **Validation**

- Are we building the right product?
- After building the prototype, the software was validated against the requirements gathered in Section 4.

12. Software Program Execution (Source Code)

The system was implemented using C++ to demonstrate the core logic and data handling.

```
#include <iostream>
#include <string>
using namespace std;

int bookId[50];
string bookTitle[50];
string bookAuthor[50];
int bookQty[50];

int countBook = 0;

bool login() {
    string user, pass;
    cout << "Username: ";
    cin >> user;
    cout << "Password: ";
    cin >> pass;

    if (user == "admin" && pass == "1234") {
        cout << "Login Successful!\n";
        return true;
    } else {
```

```
void addBook() {
    cout << "\n--- Add Book ---\n";
    cout << "Book ID: ";
    cin >> bookId[countBook];
    cin.ignore();

    cout << "Book Title: ";
    getline(cin, bookTitle[countBook]);

    cout << "Author Name: ";
    getline(cin, bookAuthor[countBook]);

    cout << "Quantity: ";
    cin >> bookQty[countBook];

    countBook++;
    cout << "Book Added Successfully!\n";
}
```



```

void displayBooks() {
    cout << "\n--- Library Books ---\n";
    for (int i = 0; i < countBook; i++) {
        cout << "ID: " << bookId[i]
            << " | Title: " << bookTitle[i]
            << " | Author: " << bookAuthor[i]
            << " | Quantity: " << bookQty[i] << endl;
    }
}

void searchBook() {
    int id;
    cout << "\nEnter Book ID: ";
    cin >> id;

    for (int i = 0; i < countBook; i++) {
        if (bookId[i] == id) {
            cout << "Book Found!\n";
            cout << "Title: " << bookTitle[i] << endl;
            cout << "Author: " << bookAuthor[i] << endl;
            cout << "Quantity: " << bookQty[i] << endl;
            return;
        }
    }
}

```

```

void issueBook() {
    int id;
    cout << "\nEnter Book ID to Issue: ";
    cin >> id;

    for (int i = 0; i < countBook; i++) {
        if (bookId[i] == id) {
            if (bookQty[i] > 0) {
                bookQty[i]--;
                cout << "Book Issued Successfully!\n";
            } else {
                cout << "Book Not Available!\n";
            }
            return;
        }
    }
    cout << "Book Not Found!\n";
}

void returnBook() {
    int id;
    cout << "\nEnter Book ID to Return: ";
    cin >> id;
}

```

```

void returnBook() {
    int id;
    cout << "\nEnter Book ID to Return: ";
    cin >> id;

    for (int i = 0; i < countBook; i++) {
        if (bookId[i] == id) {
            bookQty[i]++;
            cout << "Book Returned Successfully!\n";
            return;
        }
    }
    cout << "Book Not Found!\n";
}

int main() {
    if (!login())
        return 0;

    int choice;
    do {
        cout << "\n==== Library Management System =====\n";
        cout << "1. Add Book\n";
        cout << "2. Display Books\n";
        cout << "3. Search Book\n";

```

```

        int choice;
        do {
            cout << "\n==== Library Management System =====\n";
            cout << "1. Add Book\n";
            cout << "2. Display Books\n";
            cout << "3. Search Book\n";
            cout << "4. Issue Book\n";
            cout << "5. Return Book\n";
            cout << "6. Exit\n";
            cout << "Enter Choice: ";
            cin >> choice;

            switch (choice) {
                case 1: addBook(); break;
                case 2: displayBooks(); break;
                case 3: searchBook(); break;
                case 4: issueBook(); break;
                case 5: returnBook(); break;
                case 6: cout << "Program Exit...\n"; break;
                default: cout << "Invalid Choice!\n";
            }
        } while (choice != 6);

        return 0;

```

❖ Output

```
Username: admin  
Password: 1234  
Login Successful!
```

```
===== Library Management System =====
```

```
1. Add Book  
2. Display Books  
3. Search Book  
4. Issue Book  
5. Return Book  
6. Exit  
Enter Choice: 1
```

```
--- Add Book ---
```

```
Book ID: 321  
Book Title: cs  
Author Name: khurm  
Quantity: 4  
Book Added Successfully!
```

```
===== Library Management System =====
```

```
1. Add Book  
2. Display Books  
3. Search Book  
4. Issue Book
```